If you're running your app on Microsoft Azure, you get centralized logging for free because you can collect logs using the Azure App Service provider. Alternatively, you could send your logs to a third-party log aggregator service such as elmah.io (https://elmah.io) or Seq (https://getseq.net). You can find ASP.NET Core logging providers for each of these services on NuGet, so adding them is the same process as adding the file provider you've seen already.

Whichever providers you add, once you start running your apps in production, you'll quickly discover a new problem: the sheer number of log messages your app generates! In the next section you'll learn how to keep this under control without affecting your local development.

26.4 Changing log verbosity with filtering

In this section you'll see how to reduce the number of log messages written to the logger providers. You'll learn how to apply a base level filter, filter out messages from specific namespaces, and use logging provider-specific filters.

If you've been playing around with the logging samples, you'll probably have noticed that you get a lot of log messages, even for a single request like the one in figure 26.2: messages from the Kestrel server and messages from EF Core, not to mention your own custom messages. When you're debugging locally, having access to all that detailed information is extremely useful, but in production you'll be so swamped by noise that picking out the important messages will be difficult.

ASP.NET Core includes the ability to filter out log messages *before* they're written, based on a combination of three things: